Documentation Technique – Surveillance environnementale intelligente pour le campus

Marc Zaky, Iwen Nuss, Maxime Le Pelvé, Amaury Dubuc–Tabouy, Mathys Dubois

Année 2024-2025

Résumé

Ce document technique présente l'installation, la structure fonctionnelle et l'architecture logicielle du projet sur la surveillance environnementale intelligente pour le campus, un système de détection d'animaux basé sur un Raspberry Pi et un traitement distribué via des fonctions *serverless*. Le projet s'inscrit dans le cadre de l'étude pratique Mycélium.

Table des matières

1 Installation de l'environnement **2**
- 1.1 Préparation du Raspberry Pi 2
- 1.2 Installation de faasd (Fonctions locales) 2
- 1.3 Installation de faas-cli et Docker (sur PC) 2
- 1.4 Déploiement de fonction sur le Raspberry Pi 2

2 Architecture fonctionnelle **2**
- 2.1 Fonctions locales (Raspberry Pi) 2
- 2.2 Fonctions cloud (serveur distant) 3
- 2.3 Détails d'implémentation des fonctions principales 3

3 Base de données **5**
- 3.1 Technologie utilisée 5
- 3.2 Accès et exploitation 6

4 Notification Discord **6**

5 Conclusion **7**

1 Installation de l'environnement

1.1 Préparation du Raspberry Pi

— Flasher une image Raspberry Pi OS Lite 64 bits avec Raspberry Imager.
— Se connecter en administrateur avec `sudo -i`.

1.2 Installation de faasd (Fonctions locales)

1. Cloner le dépôt : `git clone https://github.com/openfaas/faasd`
2. Lancer l'installation : `./hack/install.sh`
3. Corriger la version de l'image Docker dans `/var/lib/faasd/docker-compose.yml` si nécessaire.
4. Redémarrer : `systemctl daemon-reload`, puis :

```
sudo systemctl status faasd
sudo systemctl status faasd-provider
sudo journalctl -u faasd --lines 40
```

5. Accéder à l'interface web : `http://<ip>:8080`

1.3 Installation de faas-cli et Docker (sur PC)

— Installer Docker : `https://docs.docker.com/engine/install/`
— Installer faas-cli : `curl -sLfS https://cli.openfaas.com | sudo sh`
— Se connecter à Docker Hub : `docker login`
— Télécharger un template : `faas-cli template store pull python3-http-debian`
— Créer une fonction :
`faas-cli new -lang python3-http-debian ma-fonction -prefix nom-utilisateur`

1.4 Déploiement de fonction sur le Raspberry Pi

— Se connecter à faas-cli :
`export OPENFAAS_URL=http://<ip>:8080`
`cat /var/lib/faasd/secrets/basic-auth-password | faas-cli login -password-stdin`
— Compiler et publier :
`faas-cli publish -f ma-fonction.yml -platform linux/arm64,linux/amd64`
— Déployer : `faas-cli deploy -f ma-fonction.yml`

2 Architecture fonctionnelle

2.1 Fonctions locales (Raspberry Pi)

— **Fonction 1 : Détection de mouvement et capture d'image**
Capture une image dès qu'un mouvement est détecté et publie un message NATS

pour lancer le pré-traitement.

— **Fonction 2 : Prétraitement YOLOv8**
Analyse l'image avec le modèle YOLOv8 et génère un fichier JSON avec les détections (classe, confiance, image, horodatage).

— **Fonction 3 : Pré-tri et filtrage**
Supprime les détections non animales et les doublons temporels. Ce programme est exécutée toutes les 24 heures et déclenche la fonction 4.

— **Fonction 4 : Transfert de données**
Envoie le dossier filtré au serveur distant. En cas d'échec : `sending_failed()`, en cas de succès : `successfull_sending()`.

2.2 Fonctions cloud (serveur distant)

— **Fonction 5 : Traitement final**
Réanalyse les images avec un modèle YOLO que nous avons entraîné et génère un nouveau JSON plus fiable.

— **Fonction 6 : Tri final**
Supprime les doublons résiduels après le traitement final.

— **Fonction 7 : Stockage et notification**
Insère les données en base SQLite et envoie une notification Discord.

2.3 Détails d'implémentation des fonctions principales

Fonction 1 : Prétraitement YOLOv8

Cette fonction tourne en continue. Elle compare les pixels entre deux instants, si un changement trop important est détecté, une photo est prise, puis le programme attend un certain temps avant de recommencer pour éviter de devoir traiter un trop grand nombre de photos du même animal.

Fonction 2 : Détection d'un mouvement et capture d'image

Application du modèle de traitement YOLOv8 sur l'image et enregistrement des données issues du traitement dans un fichier JSON qui est rempli au fil des traitements.
Etape 1 : Exécution du modèle YOLO sur l'image et récupération des informations nécéssaires (animal, effectif, certitude, heure, chemin de l'image) : `def process_image(image_path)`
Etape 2 : Sauvegarder les informations issues du traitement dans un fichier JSON qui cumule les différentes données : : `def save_detections_to_json(detection_list)`

Fonction 3 : tri (locale)

Signature : `def tri(source_folder, intervalle_secondes=15)`

Cette fonction trie les détections locales en supprimant doublons et erreurs.

Étapes : copie du dossier, nettoyage JSON, tri par heure, suppression de doublons ou d'objets non animaux.

Arguments : `source_folder`, `intervalle_secondes` (15s par défaut).

Fonction 4 : envoyer_dossier_ssh

Signature : `def envoyer_dossier_ssh(chemin_dossier_local, destination_ssh)`

Envoie un dossier via SSH avec `scp`.

Comportement associé :

— En cas d'échec : `sending_failed(source, destination)`

— En cas de succès : `successfull_sending(source)`

Fonction 5 : traitement final

Signature : `def modifier_chemin_image(destination_dossier_image, destination_fichier_json)`

Etape 1 : Changer tous les chemins des fichiers (les chemins ne sont plus les mêmes dans le serveur et dans le Raspberry).

Etape 2 : Charger le modèle YOLO que nous avons entraîné.

Etape 3 : Créer un mapping image -> heure pour conserver l'heure de chaque image afin d'éviter la perte de cette donnée causé par ce second traitement.

Etape 4 : Parcourir toutes les images du fichier, sur chacune d'elle, appliquer le traitement YOLO, récupérer les informations du traitement et écraser ces informations par dessus les anciennes du fichier JSON.

Arguments : `destination_dossier_image`, `destination_fichier_json`

Fonction 6 : tri (serveur)

Signature : `def tri(fichier_json, intervalle_secondes=15)`

Trie les données réanalysées et supprime les doublons d'images ou d'entrées JSON en fonction de la date et l'heure de détection.

Arguments : `fichier_json`, `intervalle_secondes`

Fonction 7 : stockage et notification

Insertion des données dans la base de données et envoie d'une notification discord.

Etape 1 : Chargement du fichier JSON : `load_detections_from_json(fichierDetection.json))`

Etape 2 : Création de la base de donnée et des tables si elles n'existent pas : `create_database()`

Etape 3 : Insertion des données issues du chargement du fichier JSON dans les tables de la base : `insert_data(ListDetection)`

Etape 4 : Envoie d'un message à un serveur Discord par l'intermédiaire d'un bot :

```
send_discord_notification(message)
```

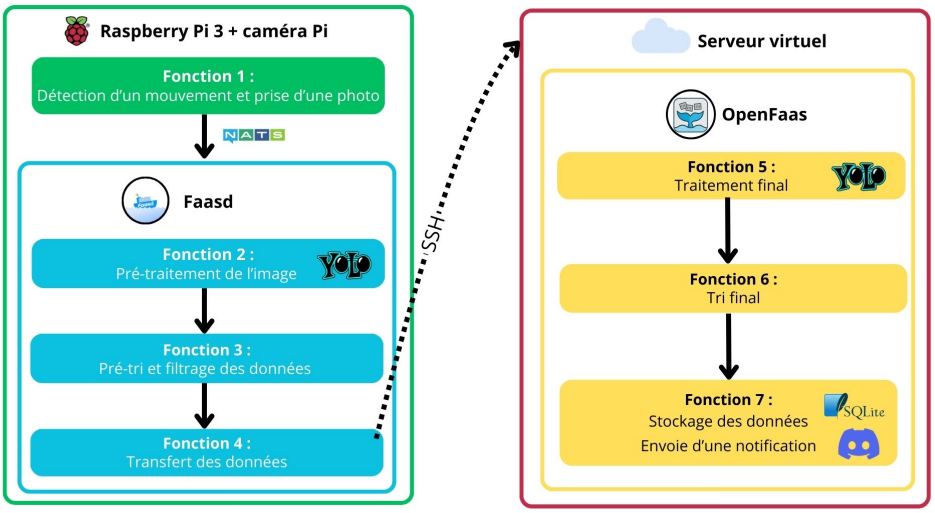

FIGURE 1 – Architecture du projet

3 Base de données

3.1 Technologie utilisée

- SGBD : SQLite
- Deux tables principales :
  - **Détections** : identifiant de la détection, classe détectée, nombre d'animaux de cette classe sur la photo, identifiant de la photo
  - **Photos** : identifiant de la photo, heure, date, nombre total d'animaux sur la photo

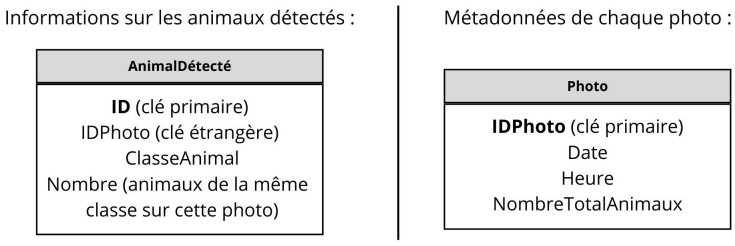

FIGURE 2 – Tables de la base de données

3.2 Accès et exploitation

- Requêtes simples via Python ou interface dédiée
- Ouverture de la base : `sqlite3 chemin/vers/taBase.db`
- Documentation SQLite : `.help`
- Sauvegarde avec : `sqlite3 database.db .dump`
- Fermeture de la base : `.quit`

Pour plus de renseignements sur la formulations des requêtes SQLite : `https://doc.ubuntu-fr.org/sqlite`

4 Notification Discord

Afin d'envoyer un message sur un serveur discord, il est nécessaire de créer un bot. Voici les instructions pour le créer.

Etape 1 : Activer le mode développeur dans les paramètres avancés de votre compte discord.
Etape 2 : Dans le portail développeur, allez dans l'onglet "Application" et cliquez sur "Nouvelle application" puis donnez un nom à votre bot et copiez l'identifiant de l'application.
Etape 3 : Dans l'onglet "Bot", cliquez sur "Ajouter un bot" afin de générer un compte bot ainsi qu'un jeton qui sera la clé d'accès du bot à Discord. **Attention :** ce jeton est à conserver en lieu sûr car sa divulgation pourrait exposer votre bot et votre serveur Discord à des risques de sécurité.
Etape 4 : Dans l'onglet "Application", configurez les autorisations que vous accordez au bot.
Etape 5 : Copiez l'URL générée en bas et ouvrez-la dans votre navigateur. Séléctionnez le bon serveur discord.
Etape 6 : Installez la bibliothèque Discord.py : `pip install -U discord.py`
Etape 7 : Il ne reste plus qu'à écrire le code souhaiter en fonction des actions que vous désirez exécuter avec le bot. Il est important de placer le jeton du bot dans un fichier `.env` pour des raisons de sécurité.

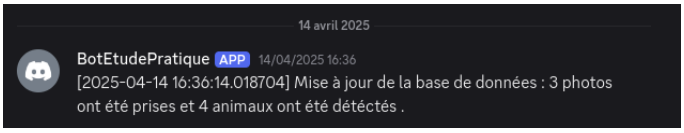

FIGURE 3 – Réception de la notification Discord

Documentation :
- Documentation Discord développer : `https://discord.com/developers/docs/intro`

— Tutoriel configuration du bot : `https://www.ionos.fr/digitalguide/serveur/know-how/creer-un-bot-discord/`
— Documentation bibliothèque `discord.py` : `https://discordpy.readthedocs.io/en/stable/`

5 Conclusion

Ce document présente l'environnement d'installation, l'architecture modulaire et le rôle de chaque fonction dans notre système. Le découplage local/cloud et l'approche *serverless* ont permis de répondre aux contraintes matérielles tout en assurant extensibilité et efficacité.